RV32IM CPU in Verilog

Design Notes: Block RAM

Jonathan Avalos

October 19, 2026

Contents

1 Overview **2**

2 Synchronous reads **2**

3 Instruction fetch **3**

4 Data port **3**

4.1 Latching the load type with the word 3

4.2 First attempt: address in MEM 4

4.3 The fix: address in EX 5

5 Address adder **6**

6 Timing **6**

7 Verification **7**

8 Next steps **7**

1 Overview

Until now `memory.v` was built from LUTs: a read returned its data in the same cycle as the address. That kept the pipeline simple, but it used logic cells for storage and can't grow much past 16 KB. The Artix-7 has 135 dedicated **block RAM** tiles (36 Kb each, about 600 KB in total) sitting unused. This step moves memory into them.

The catch is that block RAM reads are **synchronous**: the address is taken on a clock edge and the data comes out after that edge. Both the instruction fetch and the data port had to be rearranged so each address reaches the RAM one cycle early.

Status

- Memory is 4 × RAMB36 (16 KB), **LUT as memory: 0**.
- Timing: **WNS** +0.311 **ns** at 100 MHz, WHS +0.092 ns, 0 failing endpoints.
- All tests pass with no extra stall cycles.
- **Next:** Stage 3 (CSRs, traps, timer, MMIO).

Changed files

file	what changed
`memory.v`	both reads happen on the clock edge; load type, byte offset and read flag latched with the word
`core_pipe.v`	`fetch_addr`; data port driven from EX; `ex_addr` adder for `jalr` and load/store addresses

The three kinds of memory

kind	where	size	read
LUT memory (before)	inside the FPGA, logic cells	16 KB here	same cycle
block RAM (now)	inside the FPGA, dedicated tiles	∼600 KB	next edge
DDR2 (later)	separate chip on the board	128 MB	tens of cycles, needs a controller

Block RAM is enough for everything through Stage 3. DDR2 comes with the Linux stage, behind a cache that will itself be built from block RAM.

2 Synchronous reads

Vivado only uses block RAM when the read happens inside a clocked block:

```
// LUT memory: data in the same cycle
assign instr = mem[instr_addr[ADDR_BITS+1:2]];

// block RAM: data after the edge
always @(posedge clk) begin
    instr <= mem[instr_addr[ADDR_BITS+1:2]];
end
```

The memory now behaves like a register: whatever address is present at an edge is the word that comes out during the next cycle. Getting the right data at the right time means presenting the address *one cycle earlier* than before.

3 Instruction fetch

Before, IF gave the memory `pc` and IF/ID latched the instruction at the end of that cycle. Now the instruction for `pc` must already be coming out of block RAM while `pc` holds it, so the RAM has to be given the address `pc` is *about to* take:

	edge k	cycle k+1
`pc`	becomes P	holds P
block RAM	latches address P	outputs `mem[P]`
IF/ID		latches `mem[P]` at the next edge

That address is not always `pc_next`. During a load-use or mul-use stall, or while a divide holds the pipeline, `pc` keeps its value while `pc_next` is still `pc + 4`. The fetch address has to follow the IF block case by case:

```
wire [31:0] fetch_addr = rst ? RESET_PC : ex_redirect ? pc_next : (stall || ex_busy) ?
    pc : pc_next;

assign instr_addr  = fetch_addr;
```

- **Reset:** `pc` becomes `RESET_PC`, so the RAM must already be reading `RESET_PC` while `rst` is high.
- **Redirect** comes before the hold, the same order as the `if`s in the IF block.
- `fetch_addr` is declared after `ex_redirect`, `stall` and `ex_busy`, otherwise iverilog reports a use before declaration.

Figure 1 shows the timing, including a stall.

With only this change, every cycle count stayed exactly the same, which confirms the fetch lines up after stalls, redirects and reset.

4 Data port

4.1 Latching the load type with the word

A load does more than read a word: `lb`, `lh`, `lbu` and `lhu` pick a byte or halfword out of it using the byte offset and `funct3`. With a synchronous read the word comes out one edge after the address, and by then those two signals describe the next instruction. So they are latched on the same edge as the word:

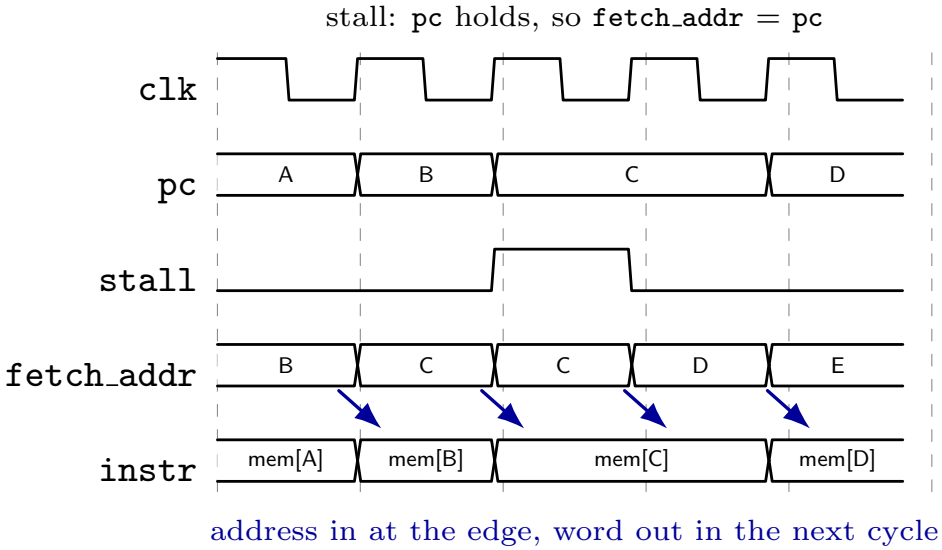


Figure 1: Synchronous fetch. Block RAM takes `fetch_addr` at each edge and outputs that word during the next cycle, which is the cycle `pc` holds the same address. During the stall `pc` holds C, so `fetch_addr` is C again and `mem[C]` stays on the output.

```
reg [31:0] word;     // read on the edge
reg [1:0]  off_r;    // byte offset
reg [2:0]  funct3_r; // load type
reg        read_r;   // was a load
wire [31:0] shifted = word >> {off_r, 3'b000};   // wanted byte/half moved to the
    bottom

always @(posedge clk) begin
    word     <= mem[data_index];
    off_r    <= mem_addr[1:0];
    funct3_r <= mem_funct3;
    read_r   <= mem_read;
end
```

The `case` that builds `mem_rdata` is unchanged except that it switches on `read_r` and `funct3_r`. It's the same rule as the multiplier's `op_r`: whatever describes an operation travels with its data. Writes are untouched and still happen on the clock edge.

4.2 First attempt: address in MEM

The first version kept the data port on the MEM-stage signals (`ex_mem_alu_result` and friends). The word then came out during **WB**, so `mem_wb_mem_rdata` was removed and `wb_data` took `mem_rdata` directly. Simulation passed, but timing failed badly:

delay	step
2.45 ns	block RAM output (`DOADO`) after the edge
	`lb`/`lh` extraction, `wb_data`
	forwarding mux into EX
	`branch_unit` less-than compare (3 `CARRY4`)
	`ex_redirect`, `pc_next`, `fetch_addr`
−0.57 ns setup	block RAM address pin
WNS −1.889 **ns**, 11 levels, 63% routing	

A load feeding a branch had to come out of one block RAM and steer the address of another, all in one cycle, starting from the RAM's slow output and ending at its early-setup address pin.

4.3 The fix: address in EX

The same idea as the fetch: give the RAM the address one cycle early. The EX-stage signals are exactly the values about to be latched into EX/MEM, so each port takes the right-hand side of its EX/MEM line:

```
assign mem_addr   = ex_addr;          // ld/st address, EX
assign mem_wdata  = ex_rs2;           // store data, forwarded
assign mem_write  = id_ex_mem_write;
assign mem_read   = id_ex_mem_read;
assign mem_funct3 = id_ex_funct3;
```

	address taken at	word comes out
first attempt	edge after MEM	during WB
fix	edge after EX	during MEM, like the LUT memory

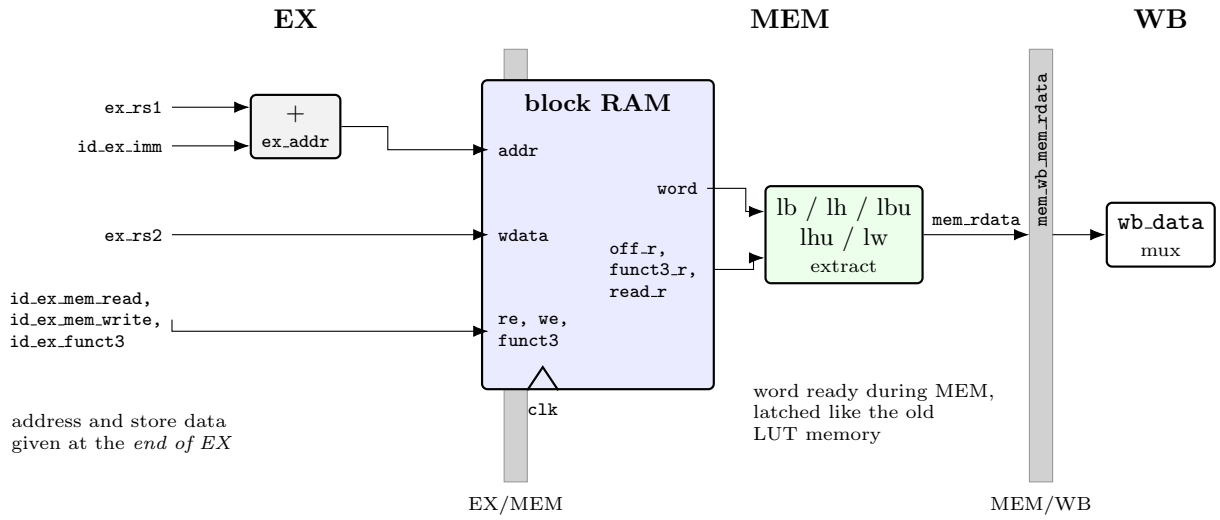


Figure 2: The data port after the fix. The address comes from `ex_addr` and the store data from the forwarded `ex_rs2`, both at the end of EX. The word and the latched load type come out during MEM, and the extracted `mem_rdata` is latched into MEM/WB.

The word is ready during MEM again (Figure 2), so `mem_wb_mem_rdata` came back and latches it at the end of MEM. The RAM's slow output now ends at a flip-flop. `memory.v` did not change for this; only what the core connects to it.

5 Address adder

With the load path cut, the worst path was `jalr` (WNS −0.228 ns, 83% routing): forwarding → the full ALU, including **4 LUT levels of its output mux** → `jalr` target → `fetch_addr` → block RAM.

But a `jalr` target, a load address and a store address are always `rs1 + imm`. They never need the ALU's shifts, compares or logic ops, or its `alu_a`/`alu_b` muxes. A dedicated adder skips all of that:

```
wire [31:0] ex_addr = ex_rs1 + id_ex_imm; // rs1 + imm, jalr & ld/st
assign ex_jalr_target = {ex_addr[31:1], 1'b0}; // bit 0 cleared
```

- For loads, stores and `jalr` the decoder always picks `rs1` and `imm` as the ALU inputs. The ALU muxes can pick `pc` (`auipc`) or `rs2` (register ops), but never for these, so `ex_addr` gives exactly the same value and nothing changes in simulation.
- It's the same idea as `ex_branch_target = id_ex_pc + id_ex_imm`, a dedicated adder for branch and `jal` targets. Real CPUs call this an *address generation unit* (AGU).
- The ALU still computes the same sum for its own result; the duplicate is only a few LUTs and `CARRY4`s.

6 Timing

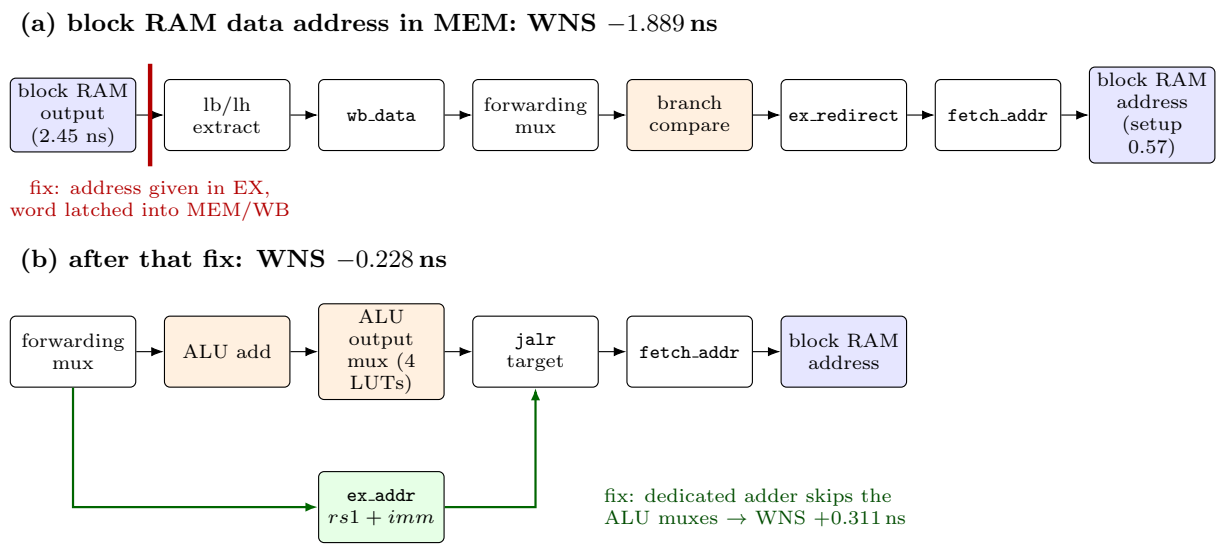


Figure 3: The two failing paths and where each was cut.

version	WNS	worst path
LUT memory (before)	+0.377 ns	branch redirect
block RAM, data address in MEM	−1.889 ns	load → branch → fetch
data address in EX	−0.228 ns	`jalr` through the ALU mux
`ex_addr` adder	**+0.311** ns	—

Utilization after the change: about 2,060 LUTs (3.3% of the chip), 1,778 flip-flops, 4 DSP48E1, 4 RAMB36, and 0 LUTs used as memory.

Slowing the clock was the fallback: a path that needs $10 - \text{WNS}$ nanoseconds sets the fastest clock at $1/(10 - \text{WNS})$, so −0.228 ns would have needed about 97.7 MHz. Fixing the paths instead keeps the full 100 MHz and leaves room for Stage 3, whose trap logic will sit on the same redirect/fetch path.

7 Verification

test	cycles	result
`test_asm.S` on the pipeline	584	48/48
`test_asm_hazards.S`	525	11/11
riscv-tests `rv32ui` + `rv32um`	—	48/48

Every count is exactly 1 lower than before block RAM (585, 526, and each riscv-test one less). Nothing ran faster or slower: the testbenches stop when the final store lands in memory, and stores now write at the end of EX instead of the end of MEM. The load and store tests of every size (`lb`, `lbu`, `lh`, `lhu`, `lw`, `sb`, `sh`, `sw`, `ld_st`, `st_ld`) all pass.

Single-cycle core

`core.v` and `top.v` use the same `memory.v`, but a single-cycle core needs the instruction in the same cycle, so they no longer work with it. Like `reg32`'s `WRITE_THROUGH`, this can be fixed with a parameter, or with a separate combinational copy of the memory for the single-cycle core.

8 Next steps

1. Program the board and check for `000D` on the LEDs.
2. **Stage 3:** CSRs (`csrrw` and friends, `mstatus`, `mtvec`, `mepc`, `mcause`), traps (`ecall`, `ebreak`, illegal instruction, `mret`), the timer interrupt, and MMIO so programs drive the LEDs, switches and UART directly.